```
import tensorflow as tf
from tensorflow.keras import layers
import matplotlib.pyplot as plt

# ======================================
# 1. Load MNIST + Normalize + Batch
# ======================================
LATENT = 100
BATCH = 128
EPOCHS = 20

(x,_),_ = tf.keras.datasets.mnist.load_data()

x = (x.astype("float32") - 127.5) / 127.5   # [-1,1]
x = x[..., None]

dataset = tf.data.Dataset.from_tensor_slices(x)
dataset = dataset.shuffle(60000).batch(BATCH, drop_remainder=True)


# ======================================
# 2. Generator
# ======================================
def build_generator():
    return tf.keras.Sequential([
        layers.Input(shape=(LATENT,)),
        layers.Dense(7*7*256, use_bias=False),
        layers.Reshape((7,7,256)),

        layers.BatchNormalization(),
        layers.LeakyReLU(),

        layers.Conv2DTranspose(128,5,2,'same',use_bias=False),
        layers.BatchNormalization(),
        layers.LeakyReLU(),

        layers.Conv2DTranspose(1,5,2,'same',activation='tanh')
    ])


# ======================================
# 3. Discriminator
# ======================================
def build_discriminator():
    return tf.keras.Sequential([
        layers.Input(shape=(28,28,1)),

        layers.Conv2D(64,5,2,'same'),
        layers.LeakyReLU(),
        layers.Dropout(0.3),

        layers.Conv2D(128,5,2,'same'),
        layers.LeakyReLU(),
        layers.Dropout(0.3),

        layers.Flatten(),
        layers.Dense(1, activation='sigmoid')
    ])


# ======================================
# 4. Loss Functions
# ======================================
bce = tf.keras.losses.BinaryCrossentropy()

def gen_loss(fake):
    return bce(tf.ones_like(fake), fake)

def disc_loss(real, fake):
    r_loss = bce(tf.ones_like(real), real)
    f_loss = bce(tf.zeros_like(fake), fake)
    return r_loss + f_loss


# ======================================
```

```
# 5. Models + Optimizers
# ======================================
G = build_generator()
D = build_discriminator()

g_opt = tf.keras.optimizers.Adam(1e-4)
d_opt = tf.keras.optimizers.Adam(1e-4)


# ======================================
# 6. Training Step
# ======================================
@tf.function
def train_step(real_imgs):

    noise = tf.random.normal([BATCH, LATENT])

    with tf.GradientTape() as gt, tf.GradientTape() as dt:

        fake_imgs = G(noise, training=True)

        real_out = D(real_imgs, training=True)
        fake_out = D(fake_imgs, training=True)

        g_loss = gen_loss(fake_out)
        d_loss = disc_loss(real_out, fake_out)

    g_grad = gt.gradient(g_loss, G.trainable_variables)
    d_grad = dt.gradient(d_loss, D.trainable_variables)

    g_opt.apply_gradients(zip(g_grad, G.trainable_variables))
    d_opt.apply_gradients(zip(d_grad, D.trainable_variables))


# ======================================
# 7-8. Show Generated Images
# ======================================
def show_images():
    noise = tf.random.normal([16, LATENT])
    preds = G(noise, training=False)

    plt.figure(figsize=(4,4))
    for i in range(16):
        plt.subplot(4,4,i+1)
        plt.imshow(preds[i,:,:,0]*127.5+127.5, cmap='gray')
        plt.axis('off')
    plt.show()


# ======================================
# Training Loop
# ======================================
for epoch in range(EPOCHS):

    for batch in dataset:
        train_step(batch)

    print("Epoch:", epoch+1)
show_images()
```

```
Epoch: 1
Epoch: 2
Epoch: 3
Epoch: 4
Epoch: 5
Epoch: 6
Epoch: 7
Epoch: 8
Epoch: 9
Epoch: 10
Epoch: 11
Epoch: 12
Epoch: 13
Epoch: 14
Epoch: 15
Epoch: 16
Epoch: 17
Epoch: 18
Epoch: 19
Epoch: 20
```

Start coding or generate with AI.